Cyber Physical Robotics

Dr. Paulo Garcia

2026

Cyber Physical Robotics

Milestones:

- Fridays in the next two weeks are holidays
  - Milestone 2 submission by the 23 October (no presentation)
  - Milestone 3 submission and presentation on November 13

Cyber Physical Robotics

Today:

- Partial orders in distributed systems

Cyber Physical Robotics

The paper that powers the Internet

- "Time, Clocks, and the Ordering of Events in a Distributed System"
- `https://www.youtube.com/watch?v=nfRouGH0oMg`

Cyber Physical Robotics

- Leslie Lamport won the Turing Award for this paper
- It describes the most fundamental problem in distributed systems

Cyber Physical Robotics

- Leslie Lamport won the Turing Award for this paper
- It describes the most fundamental problem in distributed systems
  - Just like general relativity says that time is not absolute
  - **The order of events in a distributed system is not absolute**

Cyber Physical Robotics

Setting the stage

- You have an application with several concurrent robots
- That can communicate with each other (e.g., WiFi)

Cyber Physical Robotics

Communication fails

- e.g., IP: messages may be lost

Cyber Physical Robotics

Communication fails

- e.g., IP: messages may be lost
- e.g., TCP/IP: messages may take arbitrary (positive) time

Cyber Physical Robotics

Setting the stage

- Room with 4 doors
- 2 Robots: can only look at 1 door at a time (going through door round-robin)
- 1 central server

Cyber Physical Robotics

Setting the stage

- People may come in (but never go out)
- Central server must count number of people in the room

Solution 1 (no inter-robot communication)

- When robot detects "person in door"
  - Sends message to server (+1)
  - Waits until person leaves door area
    - If it didn't wait, might double count

Solution 1 (no inter-robot communication)

- The problem:
  - While one robot is waiting
  - Other robot sees same person
    - double count

Cyber Physical Robotics

Solution 1 (no inter-robot communication)

- Clearly fails
  - We need some way to synchronize robots

Solution 2 (inter-robot communication)

- When robot detects "person in door X", if no other robot is watching that door
  - Sends message to server (+1)
  - Sends message to other robot (I got door X)
  - Waits until person leaves door area
    - If it didn't wait, might double count
  - Sends message to other robot (door X empty again)

If all messages took constant time

- Solution 2 would work

Cyber Physical Robotics

Messages:

- Robot A sees person at door X ($\rightarrow A(X)$)
- Robot A sees person leave door X ($\rightarrow A(!X)$)
- Robot B sees person at door X ($\rightarrow B(X)$)
- Robot B sees person leave door X ($\rightarrow B(!X)$)

CHULA ENG ACTNEW

Hypothetical real order of events

- $A(X) \rightarrow B(X) \rightarrow A(!X) \rightarrow B(!X)$

Cyber Physical Robotics

Possible order of events, as perceived by robot B:

- Let's draw this

Cyber Physical Robotics

Each robot potentially perceives the order of events differently

- Causality can never be broken, but is sometimes observed
  - A third robot can be told of effect before cause

Cyber Physical Robotics

As the number of concurrent systems grows:

- This effect becomes worse and worse

Cyber Physical Robotics

Where we see this problem all the time:

- The Internet
  - Views and likes
  - Bank accounts

Cyber Physical Robotics

Where you will see this problem all the time:

- Groups of robots
  - (and it's even worse....)

Cyber Physical Robotics

PAXOS

- A consensus algorithm for distributed computing
- Developed by Leslie Lamport (a.k.a., "Mr. Distributed Computing")

Cyber Physical Robotics

PAXOS - assumptions:

- Robots operate at arbitrary speeds
- Robots experience failures
- Robots may rejoin the protocol after failure
- Robots are not malicious

PAXOS - assumptions:

- Robots communicate with each other
- Messages are delivered asynchronously, and may take arbitrarily long
- Messages may be lost, re-ordered, or duplicated
- Messages are not corrupted

PAXOS - guarantees:

- For n robots, where $n = 2f + 1$
- Progress will occur if at most f robots fail simultaneously
- E.g.: for 13 robots, PAXOS withstands failure of 6 simultaneous robots

PAXOS - algorithm:

- Each robot can act in one of 3 roles at any given time:
  - Proposer
  - Acceptor
  - Learner
- Two phases, parts a and b each

PAXOS - algorithm **Phase 1a** (Prepare):

- Proposer: wants to send a message with VALUE v to all others
  - Creates a message ID n, greater than any message ID it has created before
  - Sends message with ID n to a quorum of acceptors
  - (if it cannot communicate with a quorum, waits)

PAXOS - algorithm **Phase 1b** (Promise):

- Acceptors: looks at received message with ID n
  - If n is higher than the ID of any proposal ever received, acceptor replies to proposer, promising to ignore all future messages with IDs less than n

PAXOS - algorithm **Phase 1b** (Promise):

- Acceptors: looks at received message with ID n
  - If n is higher than the ID of any proposal ever received, acceptor replies to proposer, promising to ignore all future messages with IDs less than n
  - If n is less than the ID of a previous proposal, it is ignored

PAXOS - algorithm **Phase 2a** (Accept):

- Proposer: receives promises from acceptors
  - If not enough promises received, it will either change state (see later)
  - Or eventually try again (with higher ID n)

PAXOS - algorithm **Phase 2a** (Accept):

- Proposer: receives promises from acceptors
  - Quorum of promises received, sends message with ID n and VALUE v
  - Read: "please accept this value"

PAXOS - algorithm **Phase 2b** (Accepted):

- Acceptors: receives accept request from proposer
  - **If and only if** it has not promised to accept messages with higher ID

PAXOS - algorithm **Phase 2b** (Accepted):

- Acceptors: receives accept request from proposer
  - **If and only if** it has not promised to accept messages with higher ID
  - Acceptor accepts VALUE v and send an accepted message to proposer and every learner
  - ("learners" are robots that were not part of the quorum)
  - (learners accept value if receive accepted messages from a quorum)

PAXOS - algorithm **Phase 2b** (Accepted):

- Acceptors: receives accept request from proposer
  - If they have promised to accept a higher ID
  - Just ignore the message

Cyber Physical Robotics

PAXOS is perhaps the most important algorithm for any distributed system

PAXOS by example:

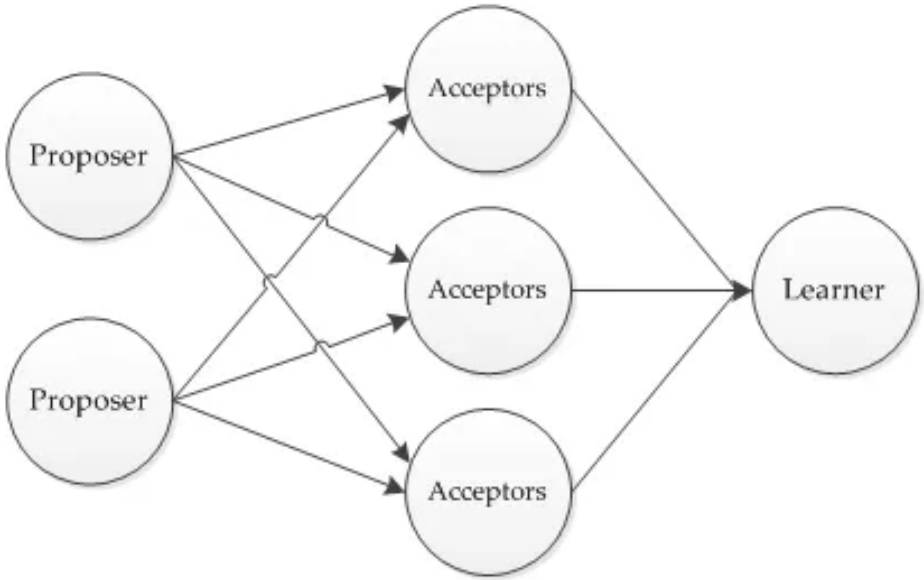


Figure 1: Basic Paxos architecture. A number of proposers make proposals to acceptors. When an acceptor accepts a value it sends the result to learner nodes.

2 proposers, both making prepare requests.
The request from proposer A reaches acceptors X and Y before the request from proposer B, but the request from proposer B reaches acceptor Z first

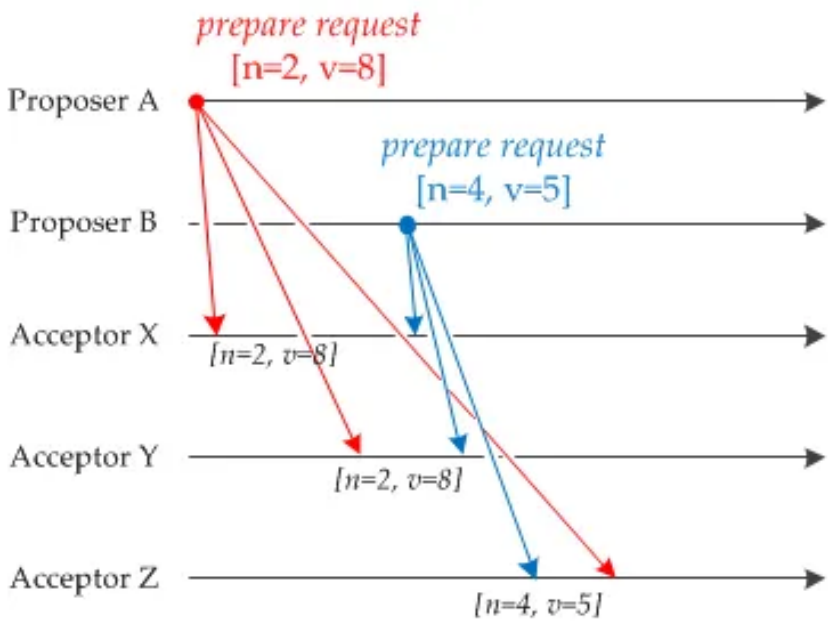


Figure 2: Proposers A and B each send prepare requests to every acceptor. In this example proposer A's request reaches acceptors X and Y first, and proposer B's request reaches acceptor Z first.

Acceptor receiving a prepare request has not seen another proposal, the acceptor responds with a prepare response which promises never to accept another proposal with a lower proposal number.

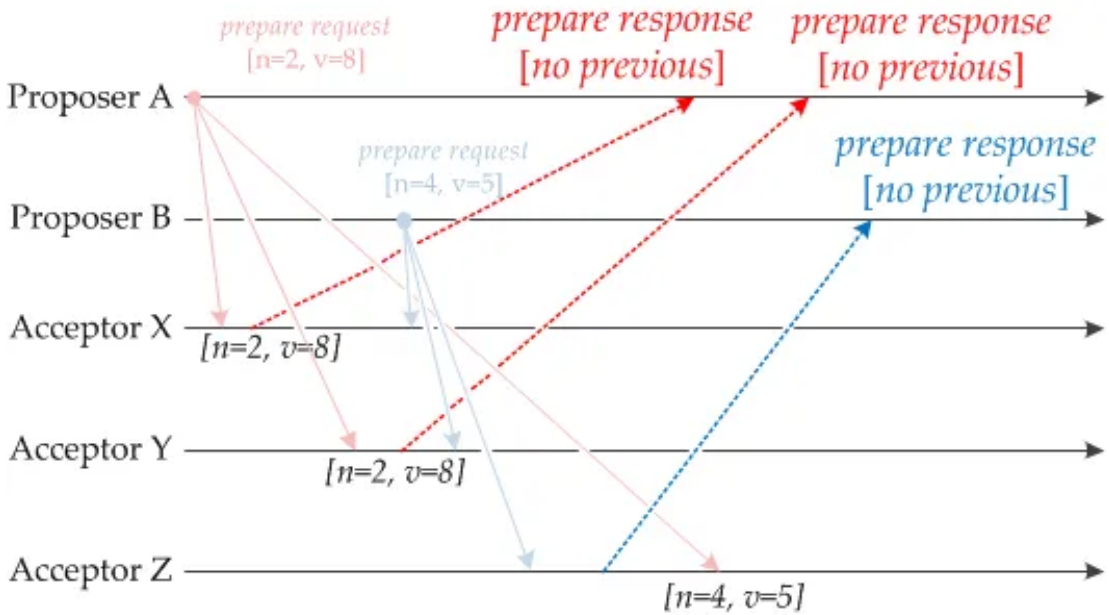


Figure 3: Paxos. Each acceptor responds to the first prepare request message that it receives.

If the acceptor has already seen a request with a higher proposal number, the prepare request is ignored, as is the case with proposer A's request to acceptor Z

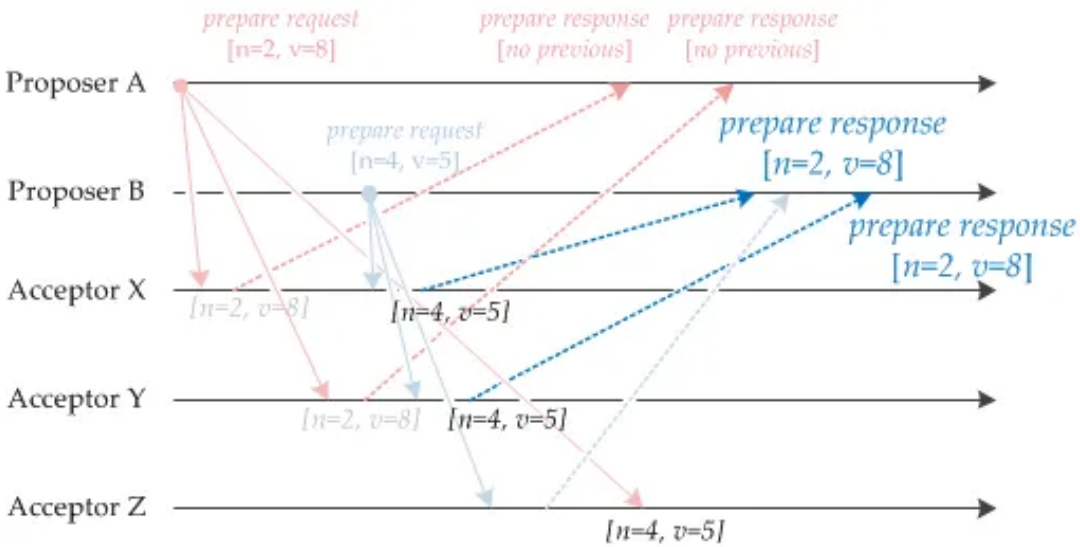


Figure 4: Acceptor Z ignores proposer A's request because it has already seen a higher numbered proposal (4 > 2). Acceptors X and Y respond to proposer B's request with the previous highest request that they acknowledged, and a promise to ignore any lower numbered proposals.

If the acceptor has not seen a higher numbered request, it again promises to ignore any requests with lower proposal numbers, and sends back the highest-numbered proposal that it has accepted along with the value of that proposal

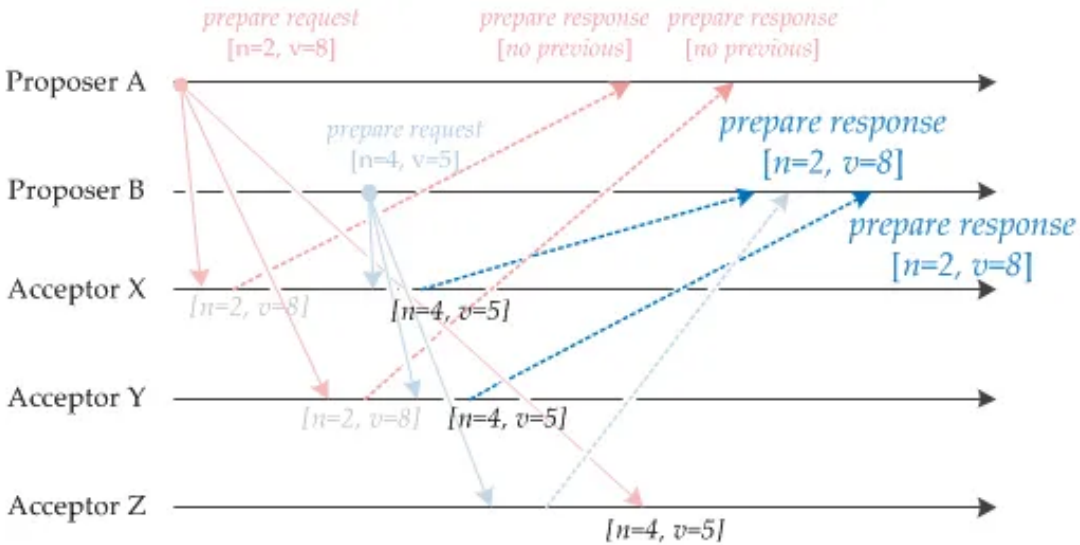


Figure 4: Acceptor Z ignores proposer A's request because it has already seen a higher numbered proposal (4 > 2). Acceptors X and Y respond to proposer B's request with the previous highest request that they acknowledged, and a promise to ignore any lower numbered proposals.

proposer A only received responses indicating that there were no previous proposals, it sends an accept request to every acceptor with the same proposal number and value as its initial proposal (n=2, v=8)

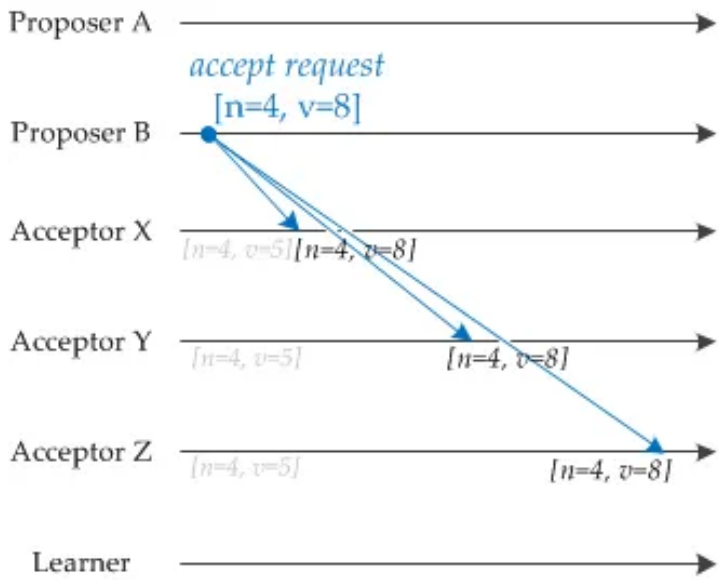


Figure 5: Proposer B sends an accept request to each acceptor, with its previous proposal number (4), and the value of the highest numbered proposal it has seen (8, from [n=2, v=8

these requests are ignored by every acceptor because they have all promised not to accept requests with a proposal number lower than 4 (in response to the prepare request from proposer B)

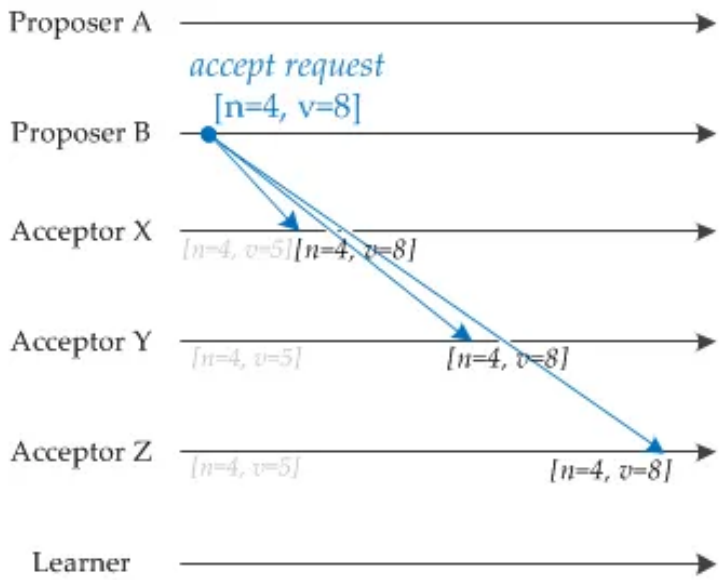


Figure 5: Proposer B sends an accept request to each acceptor, with its previous proposal number (4), and the value of the highest numbered proposal it has seen (8, from [n=2, v=8

Proposer B sends an accept request to each acceptor containing the proposal number it previously used (n=4) and the value associated with the highest proposal number among the prepare response messages it received (v=8)

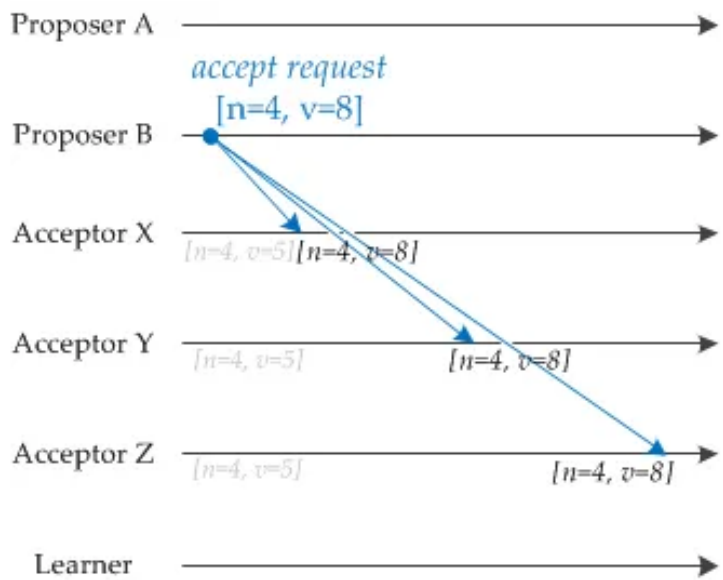
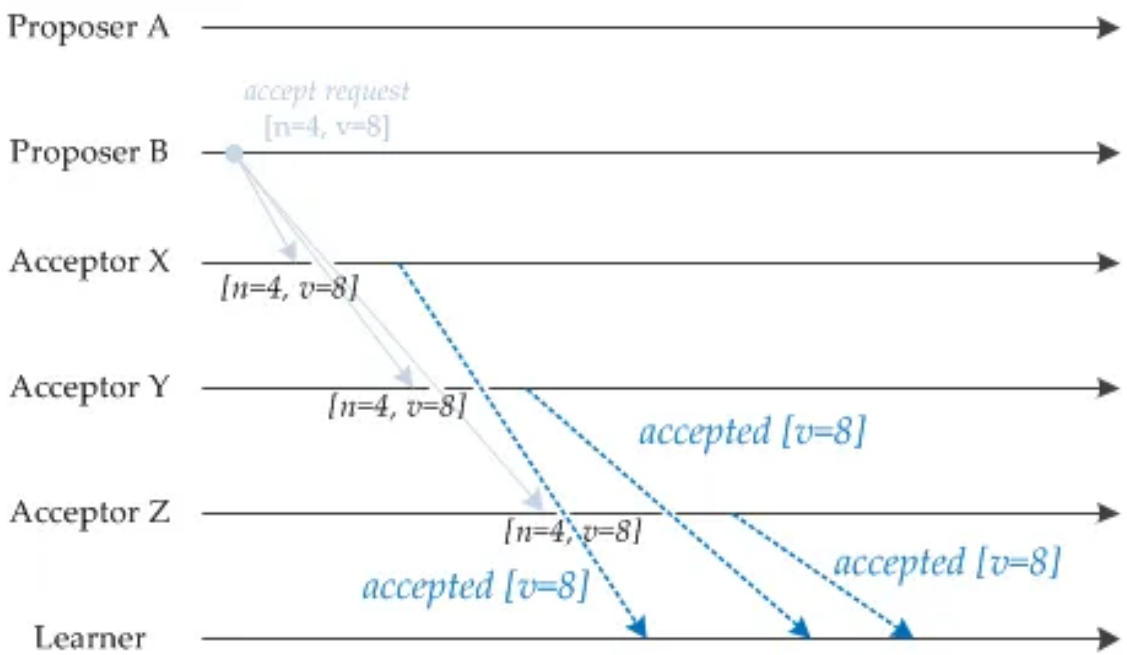


Figure 5: Proposer B sends an accept request to each acceptor, with its previous proposal number (4), and the value of the highest numbered proposal it has seen (8, from [n=2, v=8

acceptor receives an accept request for a higher or equal proposal number than it has already seen, it accepts and sends a notification to every learner node. A value is chosen by the Paxos algorithm when a learner discovers that a majority of acceptors have accepted a value



Cyber Physical Robotics

Lamport explaining Paxos

- Discussion on Paxos
- `https://www.youtube.com/watch?v=RFkzp7WXAT4`

Cyber Physical Robotics

Some concluding remarks

- Cyber Physical Systems is a young field
- Cyber Physical Robotics is even younger

Cyber Physical Robotics

Some concluding remarks

- Fundamentally, CPR is about the application of distributed systems theory to robotic systems

Cyber Physical Robotics

Some concluding remarks

- Fundamentally, CPR is about the application of distributed systems theory to robotic systems
- **Plenty of work to do in this field**